

Human Detection for Search and Rescue: Onboard vs Edge vs Cloud Inference

CENTRAL INSTITUTE OF TECHNOLOGY KOKRAJHAR

(Deemed to be University under MoE, Govt. of India)



B.Tech 6th Semester

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Presented By:

GOPICHAND SABBAVARAPU (202302023118)

KULDEEP DAS (202302021062)

ARUN DAIMARY (202302021080)

Supervised By:

Dr. Ranjan Patowary

(Assistant Professor)

Dept. Of Computer Science & Engineering

Table of Contents

01

Introduction

02

Problem Statement

03

Objective

04

Existing Work

05

Proposed System
Architectures

06

Experimental setup

07

Dataset Preparation

08

Common Inference Pipeline

09

Performance Evaluation
Metrics

10

Conclusion

11

Future Work

12

Reference

Introduction

- Search and Rescue (SAR) operations are emergency missions used to locate and rescue missing, injured, or trapped people during disasters and accidents.
- SAR operations are important because quick rescue can save lives during emergencies such as floods, earthquakes, landslides, and forest incidents.
- Camera-mounted quadcopters (drones) are widely used in SAR operations because they can quickly monitor large and dangerous areas and capture aerial views from hard-to-reach locations.
- Low latency is important in SAR systems because faster data processing and response help rescue teams identify victims more quickly during critical situations.
- Using drones helps improve search efficiency and reduces risk for rescue teams.

Problem Statement

Problem Statement

- This project deploys a human detection algorithm in three different scenarios: Onboard Inference, Edge Inference, and Cloud Inference for Search and Rescue (SAR) operations.
- Comparison Parameters:
 - Inference Latency
 - Transmission Latency
 - Power Consumption

Objectives

- Prepare a VisDrone-based dataset by merging pedestrian and people classes into a single human class for training and evaluation.
- Train a YOLOv8n model and deploy it in ONNX format.
- Implement three inference architectures for human detection: Onboard Inference, Edge Inference, and Cloud Inference.
- Compare the three architectures based on inference latency, transmission delay, and power consumption.
- Analyse the performance and identify the most suitable method for different SAR scenarios.

Existing Work

SI	Paper Title	Authors & Year	Research Focus
01	Autonomous Human Detection using Drones in Search and Rescue Operations	Dasu Vaman Ravi Prasad et al., 2024	SAR Human Detection Drone-Based Rescue System
02	AERO: AI-Enabled Remote Sensing Observation with Onboard Edge Computing in UAVs	Dasu Vaman Ravi Prasad et al., 2024	Real-Time Human Detection Drone-Based SAR System
03	DroneTrack: Cloud-Based Real-Time Object Tracking Using Unmanned Aerial Vehicles Over the Internet	Anis Koubaa et al., 2018	Cloud-Based UAV Tracking Real-Time Object Tracking
04	A Survey on the Convergence of Edge Computing and AI for UAVs: Opportunities and Challenges	Patrick McEnroe et al., 2023	AI-Enabled Edge Computing Low-Latency UAV Processing
05	YOLO-Based UAV Technology: A Review of the Research and Its Applications	Chunling Chen et al., 2022	YOLO-Based Object Detection Real-Time UAV Surveillance

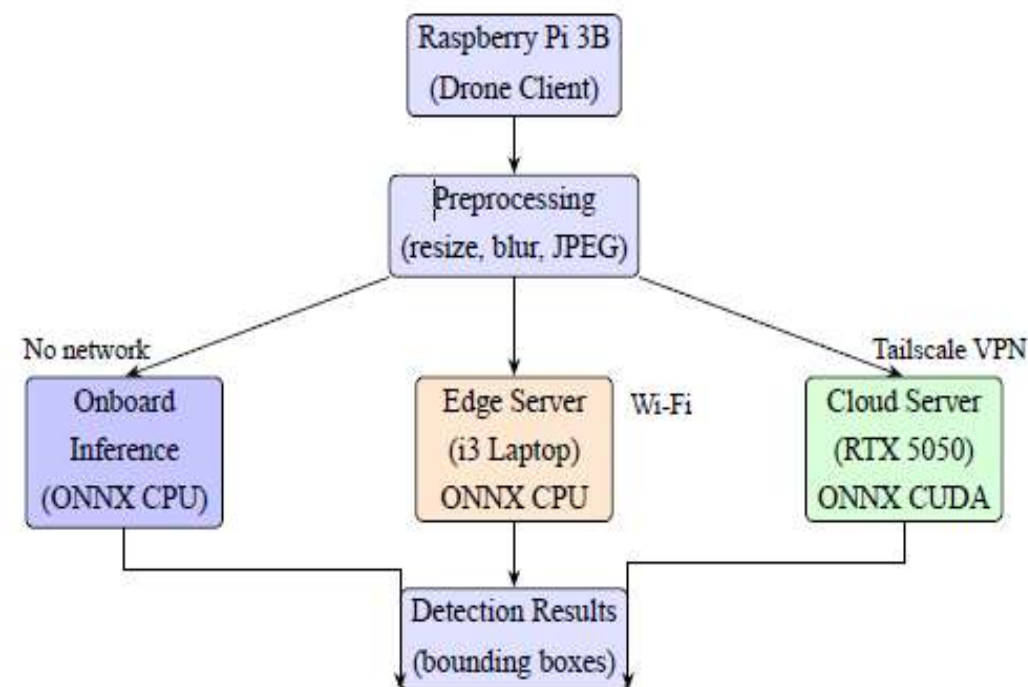
Existing Work(cont...)

06	Detection of Humans in Search and Rescue Operations Using Ensemble Learning and YOLOv9	Ananya Basireddy et al., 2024	Hybrid YOLOv9–EfficientDet Framework Thermal and RGB-Based SAR Detection
07	Human Detection in UAV Imagery Using Deep Learning: A Review	Débora Paula Simões et al., 2025	Deep Learning-Based Human Detection UAV Image Analysis Challenges
08	Automatic Person Detection in Search and Rescue Operations Using Deep CNN Detectors	Saša Sambolek et al., 2020	CNN-Based Human Detection Aerial SAR Image Processing
09	Integrating AI with Edge Computing and Cloud Services for Real-Time Data Processing and Decision Making	Md Emran Hossain et al., 2023	Edge–Cloud AI Integration Real-Time Decision Making
10	Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding	Song Han et al., 2016	Deep Learning Model Compression Efficient Edge Device Deployment

Proposed System Architectures

We propose a system architecture consisting of Onboard, Edge, and Cloud inference

- Onboard — Inference runs directly on the Raspberry Pi's ARM CPU; no network required, the image never leaves the drone.
- Edge — Image is captured on the Pi and transmitted over local Wi-Fi to a nearby laptop, where inference runs on the CPU.
- Cloud — Image is captured on the Pi and sent over the internet via Tailscale VPN to a GPU-equipped laptop simulating a cloud server, where inference runs on the GPU.



Experimental setup

Onboard setup



Edge setup



Cloud setup

Dataset Preparation

Here are the updated points:

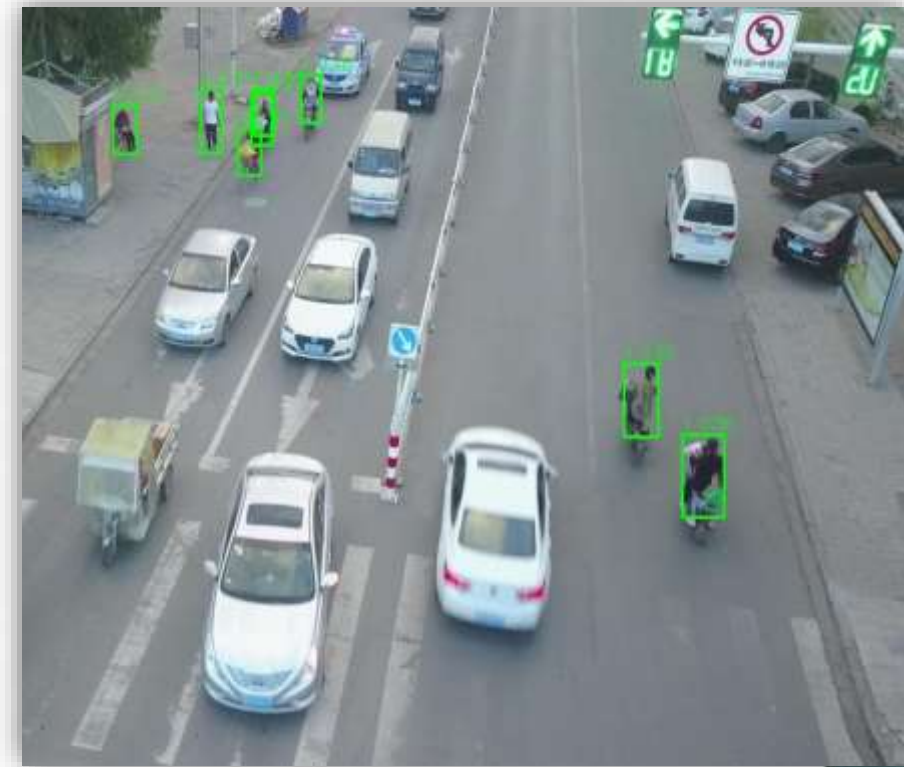
- Downloaded the VisDrone dataset — 10,209 drone images across 14 cities, captured at 5m–100m altitude with 10 object classes in YOLO format.
- Since we only needed humans, we filtered labels to keep Class 0 (pedestrian) and Class 1 (people), dropping all other 8 classes.
- Both classes were merged into a single Class 0: Human, turning it into a straightforward single-class detection task.
- Empty label files were retained as hard negatives to help the model avoid false detections in human-free frames.
- Used the predefined train/val split of 6,471 and 548 images, and sampled 290 images from the test split due to Raspberry Pi time constraints.

Common Inference Pipeline

Each image is preprocessed on the Raspberry Pi before being sent for inference.

Steps:

- Input image is read from Raspberry Pi 3
- Resized to 640×640 pixels to match model input size
- Laplacian variance is computed to check image sharpness
- If variance is too low the frame is dropped as too blurry
- Otherwise denoising is applied
- Preprocessed image is sent to the inference device (Pi / Laptop / Cloud)
- Pixel values are normalized to 0–1 range just before being fed into the YOLOv8n model
- YOLOv8n outputs bounding boxes with confidence scores for detected humans

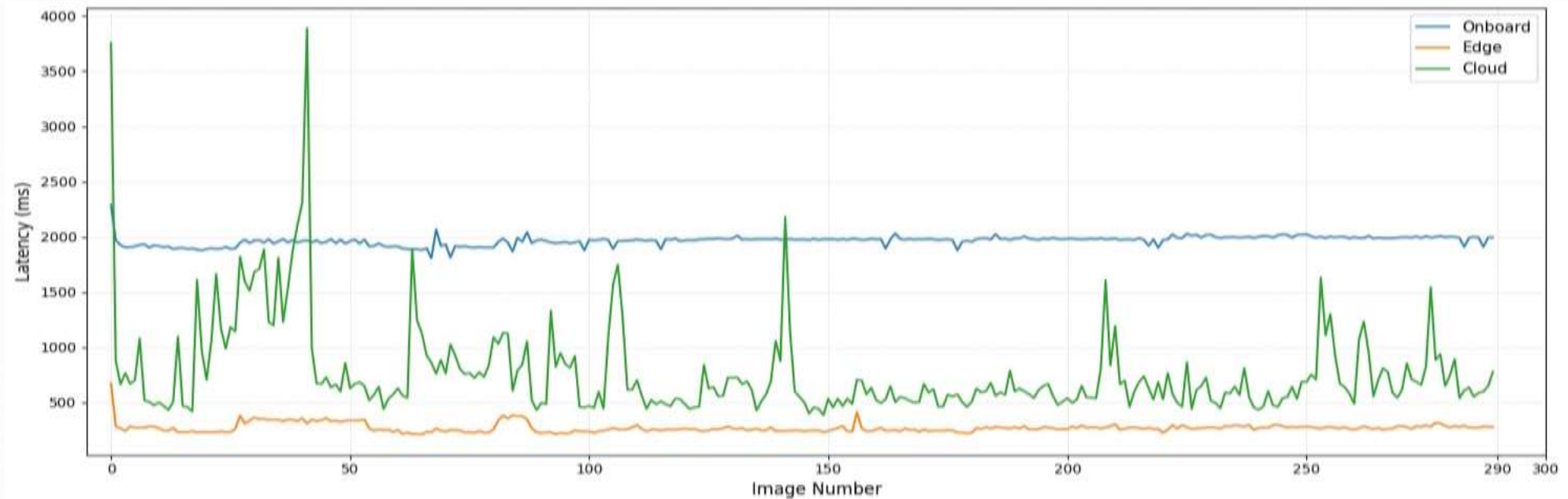


Performance Evaluation Metrics

Total Latency Comparison

- Total latency represents the complete execution time required for preprocessing, transmission, and inference.

Scenario	Total Latency
Edge	267.95 ms
Cloud	766.63 ms
Onboard	1963.75 ms



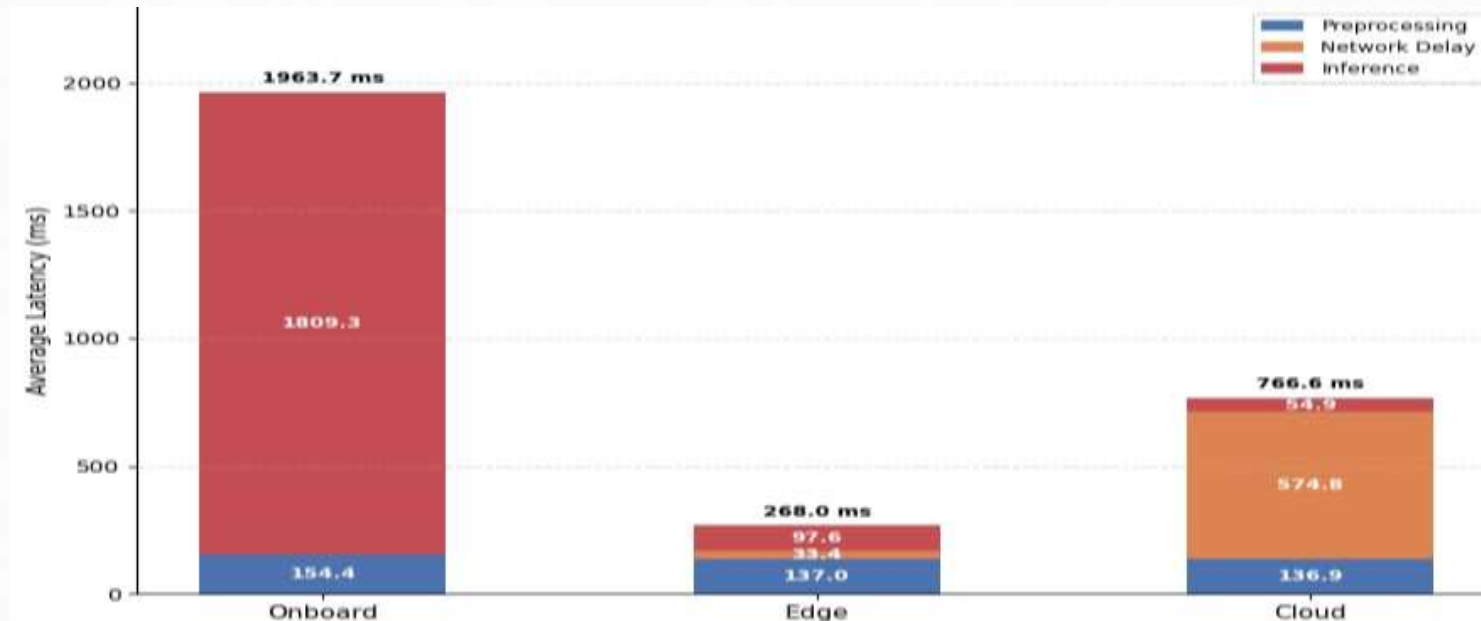
Performance Evaluation Metrics (cont...)

Latency Components Analysis

- Latency is divided into preprocessing latency, transmission delay, and inference latency.

Component Results

Component	Edge	Cloud	Onboard
Preprocessing	137 ms	137 ms	154 ms
Transmission	33 ms	579 ms	0 ms
Inference	98 ms	55 ms	1809 ms



Performance Evaluation Metrics (cont...)

Power Analysis of Three Inference Scenarios

- Edge Scenario: Lowest power consumption (54 mAh) and highest energy efficiency
- Onboard Scenario: Highest power consumption (145 mAh) due to continuous local processing
- Cloud Scenario: Moderate power consumption (99 mAh) because of network and VPN communication overhead

Scenario	Total Energy (mAh)	Run Time	Avg Power (W)	Relative to Edge
Onboard	145	12 min 00 s	3.63	2.69×
Edge	54	3 min 45 s	4.32	1.00× (baseline)
Cloud	99	11 min 00 s	2.70	1.83×

Conclusion

- Edge scenario achieved the lowest total latency due to low transmission delay and fast inference time.
- Onboard scenario had no transmission delay but very high inference time on the Raspberry Pi.
- Cloud scenario had fast inference processing but high transmission delay because of VPN/internet communication.
- Edge scenario consumed the least power, making it more battery efficient for drones.
- Therefore, the Edge inference scenario is the most suitable architecture for real-time SAR drone operations.

Future Work

- **Real-time video streaming:** Extend the current image-based pipeline to support continuous real-time video streaming and live frame-by-frame inference for drone-based surveillance and search-and-rescue operations.
- **Model optimisation:** Develop a lightweight model using techniques such as pruning, layer fusion, and INT8 quantization to reduce model size, memory usage, and inference latency for efficient deployment on onboard, edge, and cloud systems.
- **Improved recall and accuracy:** Investigate advanced data augmentation, hyperparameter tuning, additional training datasets, and improved YOLO-based architectures to enhance human detection recall and overall detection accuracy.

Reference

- **A. Koubaa et al.** — Proposed AI-enabled onboard edge computing for UAV systems with reduced communication dependency.
- **P. McEnroe et al.** — Presented a survey on edge AI deployment for UAV systems with low-latency processing.
- **A. Basireddy et al.** — Developed a human detection system for SAR operations using YOLOv9 and ensemble learning.
- **S. Sambolek and M. Ivasić-Kos** — Proposed deep CNN-based person detection for aerial rescue applications.
- **S. Han et al.** — Introduced deep compression techniques such as pruning and quantisation for efficient AI deployment.
- **J. Redmon et al.** — Introduced the YOLO framework for real-time object detection.
- **A. Bochkovskiy et al.** — Developed YOLOv4 with improved speed and detection accuracy.
- **G. Jocher et al.** — Developed YOLOv5 for lightweight edge-based object detection.
- **Ultralytics** — Introduced YOLOv8 with improved inference and deployment capabilities.
- **M. Tan and Q. V. Le** — Proposed EfficientDet for scalable and efficient object detection



Thank You